

LAPORAN HASIL PRAKTIKUM ALGORITMA DAN STRUKTUR DATA
JOBSHEET V
SORTING



Disusun oleh: Muhammad Toriq Januarsyah
Kelas TI-1H
254107020233

PROGRAM STUDI D-IV TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

5.2 Praktikum 1 - Mengimplementasikan Sorting menggunakan object

5.2.1 Langkah Praktikum 1

a. SORTING – BUBBLE SORT

1. Buat folder baru dengan nama Praktikum05.
2. Buat class Sorting<No Presensi>, kemudian tambahkan atribut sebagai berikut:

```
public class Sorting {  
    int [] data;  
    int jumData;  
}
```

3. Buatlah konstruktor dengan parameter Data[] dan jmlDat

```
Sorting (int Data[], int jmlDat){  
    jumData=jmlDat;  
    data=new int[jmlDat];  
    for (int i=0; i<jumData; i++){  
        data[i]= Data[i];  
    }  
}
```

4. Buatlah method **bubbleSort** bertipe void dan deklarasikan isinya menggunakan algoritma Bubble Sort.

```
void bubbleSort(){  
    int temp=0;  
    for (int i=0; i<jumData-1; i++){  
        for (int j=1; j<jumData-i; j++){  
            if (data[j-1]>data[j]){  
                temp=data[j];  
                data[j]=data[j-1];  
                data[j-1]=temp;  
            }  
        }  
    }  
}
```

5. Buatlah method tampil bertipe void dan deklarasikan isi method tersebut.

```
void tampil(){  
    for (int i=0; i<jumData;i++){  
        System.out.print(data[i]+" ");  
    }  
    System.out.println();  
}
```

```

J Sorting17.java U
Praktikum_05 > J Sorting17.java > Sorting17
1  package Praktikum_05;
2
3  public class Sorting17 {
4      int[] data;
5      int jumData;
6
7
8  Sorting17 (int Data[], int jmlDat) {
9      jumData = jmlDat;
10     data = new int[jmlDat];
11     for (int i = 0; i < jumData; i++) {
12         data[i] = Data[i];
13     }
14 }
15
16 void bubblesort() {
17     int temp = 0;
18     for (int i = 0; i < jumData - 1; i++) {
19         for (int j = 0; j < jumData - i; j++) {
20             if (data[j-1] > data[j]) {
21                 temp = data[j];
22                 data[j] = data[j-1];
23                 data[j-1] = temp;
24             }
25         }
26     }
27 }
28
29 void tampilData() {
30     for (int i = 0; i < jumData; i++) {
31         System.out.print(data[i] + " ");
32     }
33     System.out.println();
34 }
35 }

```

6. Buat class SortingMain<No Presensi> kemudian deklarasikan array dengan nama a[] kemudian isi array tersebut

```
int a[] = {20, 10, 2, 7, 12};
```

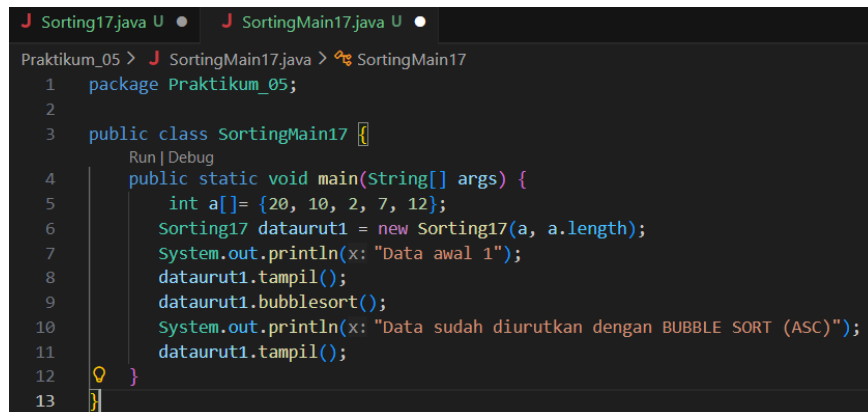
7. Buatlah objek baru dengan nama dataurut1 yang merupakan instansiasi dari class Sorting, kemudian isi parameternya

```
Sorting dataurut1 = new Sorting(a, a.length);
```

8. Lakukan pemanggilan method bubbleSort dan tampil

```
System.out.println(x:"Data awal 1");
dataurut1.tampil();
dataurut1.bubbleSort();
System.out.println(x:"Data sudah diurutkan dengan BUBBLE SORT (ASC)");
dataurut1.tampil();
```

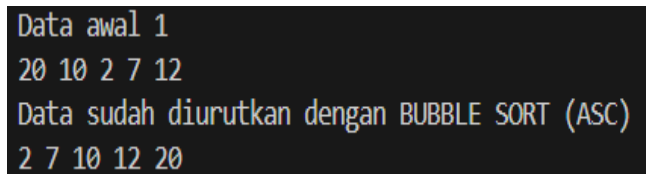
9. Jalankan program, dan amati hasilnya!



```
1 package Praktikum_05;
2
3 public class SortingMain17 {
4     public static void main(String[] args) {
5         int a[] = {20, 10, 2, 7, 12};
6         Sorting17 dataurut1 = new Sorting17(a, a.length);
7         System.out.println(x: "Data awal 1");
8         dataurut1.tampil();
9         dataurut1.bubblesort();
10        System.out.println(x: "Data sudah diurutkan dengan BUBBLE SORT (ASC)");
11        dataurut1.tampil();
12    }
13 }
```

5.2.2 Verifikasi Hasil Percobaan

Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT (ASC)
2 7 10 12 20



```
Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT (ASC)
2 7 10 12 20
```

b. SORTING – SELECTION SORT

1. Pada class **Sorting**<No Presensi> yang sudah dibuat di praktikum sebelumnya tambahkan method **SelectionSort** yang mengimplementasikan pengurutan menggunakan algoritma selection sort.

```
void SelectionSort(){
    for (int i=0; i<jumData-1; i++){
        int min=i;
        for (int j=i+1; j<jumData; j++){
            if(data[j]<data[min]){
                min=j;
            }
        }
        int temp=data[i];
        data[i]=data[min];
        data[min]=temp;
    }
}
```



```
3 public class Sorting17 {
34 }
35
36 void selectionSort() {
37     for (int i = 0; i < jumData - 1; i++) {
38         int min = i;
39         for (int j = i+1; j < jumData; j++) {
40             if(data[j]<data[min]) {
41                 min=j;
42             }
43         }
44         int temp = data[i];
45         data[i] = data[min];
46         data[min] = temp;
47     }
48 }
```

2. Deklarasikan array dengan nama **b[]** pada kelas **SortingMain**<No Presensi> kemudian isi array tersebut

```
int b[] = {30, 20, 2, 8, 14};
```

3. Buatlah objek baru dengan nama **dataurut2** yang merupakan instansiasi dari class **Sorting**, kemudian isi parameternya

```
Sorting dataurut2 = new Sorting(b, b.length);
```

4. Lakukan pemanggilan method **SelectionSort** dan **tampil**

```
System.out.println(x:"Data awal 2");
dataurut2.tampil();
dataurut2.SelectionSort();
System.out.println(x:"Data sudah diurutkan dengan SELECTION SORT (ASC)");
dataurut2.tampil();
```

5. Jalankan program dan amati hasilnya!

```
package Praktikum_05;

public class SortingMain17 {
    public static void main(String[] args) {
        int a[] = {20, 10, 2, 7, 12};
        int b[] = {30, 20, 2, 8, 14};
        Sorting17 dataurut1 = new Sorting17(a, a.length);
        System.out.println(x:"Data awal 1");
        dataurut1.tampil();
        dataurut1.bubblesort();
        System.out.println(x:"Data sudah diurutkan dengan BUBBLE SORT (ASC)");
        dataurut1.tampil();
        Sorting17 dataurut2 = new Sorting17(b, b.length);
        System.out.println(x:"Data awal 2");
        dataurut2.tampil();
        dataurut2.selectionSort();
        System.out.println(x:"Data sudah diurutkan dengan SELECTION SORT (ASC)");
        dataurut2.tampil();
    }
}
```

5.2.3 Verifikasi Hasil Percobaan

```
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30
```

```
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30
```

c. SORTING – INSERTION SORT

1. Pada class **Sorting**<No Presensi> yang sudah dibuat di praktikum sebelumnya tambahkan method **insertionSort** yang mengimplementasikan pengurutan menggunakan algoritma insertion sort.

```
void insertionSort(){
    for (int i=1; i<=data.length-1; i++){
        int temp=data[i];
        int j=i-1;
        while (j>=0 && data[j]>temp){
            data[j+1]=data[j];
            j--;
        }
        data[j+1]=temp;
    }
}
```

```
public class Sorting17 {
    void selectionSort() {
        int temp = data[i];
        data[i] = data[min];
        data[min] = temp;
    }

    void insertionSort() {
        for (int i = 1; i <= data.length-1; i++) {
            int temp = data[i];
            int j = i-1;
            while (j >= 0 && data[j] > temp) {
                data[j+1] = data[j];
                j--;
            }
            data[j+1] = temp;
        }
    }
}
```

2. Deklarasikan array dengan nama **c[]** pada kelas **SortingMain**<No Presensi> kemudian isi array tersebut

```
int c[] = {40, 10, 4, 9, 3};
```

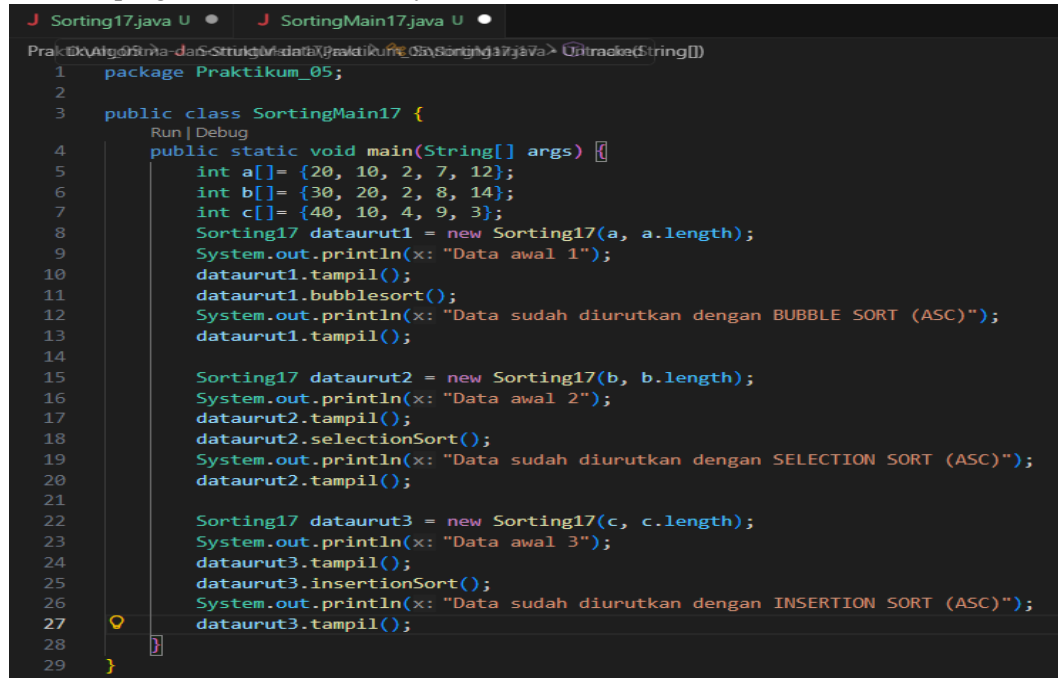
3. Buatlah objek baru dengan nama **dataurut3** yang merupakan instansiasi dari class **Sorting**, kemudian isi parameternya

```
Sorting dataurut3 = new Sorting(c, c.length);
```

4. Lakukan pemanggilan method **insertionSort** dan **tampil**

```
System.out.println(x:"Data awal 3");
dataurut3.tampil();
dataurut3.insertionSort();
System.out.println(x:"Data sudah diurutkan dengan INSERTION SORT (ASC)");
dataurut3.tampil();
```

5. Jalankan program dan amati hasilnya!



```
1 package Praktikum_05;
2
3 public class SortingMain17 {
4     public static void main(String[] args) {
5         int a[] = {20, 10, 2, 7, 12};
6         int b[] = {30, 20, 2, 8, 14};
7         int c[] = {40, 10, 4, 9, 3};
8         Sorting17 dataurut1 = new Sorting17(a, a.length);
9         System.out.println(x: "Data awal 1");
10        dataurut1.tampil();
11        dataurut1.bubblesort();
12        System.out.println(x: "Data sudah diurutkan dengan BUBBLE SORT (ASC)");
13        dataurut1.tampil();
14
15        Sorting17 dataurut2 = new Sorting17(b, b.length);
16        System.out.println(x: "Data awal 2");
17        dataurut2.tampil();
18        dataurut2.selectionSort();
19        System.out.println(x: "Data sudah diurutkan dengan SELECTION SORT (ASC)");
20        dataurut2.tampil();
21
22        Sorting17 dataurut3 = new Sorting17(c, c.length);
23        System.out.println(x: "Data awal 3");
24        dataurut3.tampil();
25        dataurut3.insertionSort();
26        System.out.println(x: "Data sudah diurutkan dengan INSERTION SORT (ASC)");
27        dataurut3.tampil();
28    }
29 }
```

5.2.4 Verifikasi Hasil Percobaan

```
Data awal 3
40 10 4 9 3
Data sudah diurutkan dengan INSERTION SORT (ASC)
3 4 9 10 40
```

```
Data awal 3
40 10 4 9 3
Data sudah diurutkan dengan INSERTION SORT (ASC)
3 4 9 10 40
```

5.2.5 Pertanyaan!

1. Jelaskan fungsi kode program berikut

```
if (data[j-1]>data[j]){
    temp=data[j];
    data[j]=data[j-1];
    data[j-1]=temp;
}
```

- Potongan kode tersebut adalah inti dari mekanisme pertukaran (Swap) pada algoritma Bubble Sort.

Bagian `if (data[j] < data[j - 1])` berfungsi sebagai pengecek kondisi. Jika angka di sebelah kanan (`data[j]`) ternyata lebih kecil dari angka di sebelah kirinya (`data[j-1]`),

maka urutannya dianggap belum benar. Variabel temp di sini berfungsi sebagai "wadah penampung sementara". Kita perlu menyimpan nilai salah satu angka di sana agar saat kita menimpa nilainya, angka asli tersebut tidak hilang. Secara keseluruhan, kode ini bertugas untuk menggeser angka yang lebih kecil ke arah kiri dan angka yang lebih besar ke arah kanan secara bertahap.

2. Tunjukkan kode program yang merupakan algoritma pencarian nilai minimum pada selection sort!

- Berdasarkan struktur algoritma Selection Sort, bagian yang berfungsi khusus untuk mencari atau menyeleksi nilai terkecil (minimum) adalah perulangan dalam (inner loop) yang membandingkan setiap elemen dengan nilai yang sementara dianggap terkecil.

potongan kode :

```
int min = i;
for ( int j = i+1; j < jumData; j++) {
    if(data[j]<data[min]) {
        min=j;
```

3. Pada Insertion sort , jelaskan maksud dari kondisi pada perulangan

```
while (j >= 0 && data[j] > temp)
```

- Perulangan while tersebut adalah inti dari proses pergeseran data pada Insertion Sort untuk menentukan posisi yang tepat bagi angka yang sedang diperiksa (temp). Kondisi ini memiliki dua syarat yang harus dipenuhi sekaligus agar perulangan tetap berjalan:

1. Syarat Pertama ($j \geq 0$):

Memastikan bahwa indeks j (angka di sebelah kiri) tidak menyentuh angka di bawah 0. Tanpa syarat ini, program akan eror (ArrayIndexOutOfBoundsException) karena mencoba mengakses posisi data yang tidak ada.

2. Syarat Kedua ($\text{data}[j] > \text{temp}$):

Program akan terus melihat ke kiri selama angka yang ada di sebelah kiri ($\text{data}[j]$) nilainya lebih besar daripada angka yang sedang kita bawa (temp). Selama kedua kondisi itu benar, berarti angka di sebelah kiri masih "terlalu besar" dan harus digeser ke kanan untuk memberi ruang. Perulangan akan langsung berhenti jika:

- 1) Ditemukan angka yang lebih kecil atau sama dengan temp (berarti posisi sisip sudah ketemu).
- 2) Atau, sudah sampai ke ujung paling kiri (indeks 0).

4. Pada Insertion sort, apakah tujuan dari perintah

```
data[j+1] = data[j];
```

- Tujuan utama dari perintah tersebut adalah untuk menggeser elemen yang lebih besar ke posisi sebelah kanannya guna menciptakan ruang kosong (celah) bagi angka yang sedang diurutkan (temp).

5.3 Praktikum 2- (Sorting Menggunakan Array of Object)

5.3.1 Langkah Praktikum 2 - Mengurutkan Data Mahasiswa Berdasarkan IPK (Bubble Sort)

5.3.2 Langkah-langkah Praktikum 2

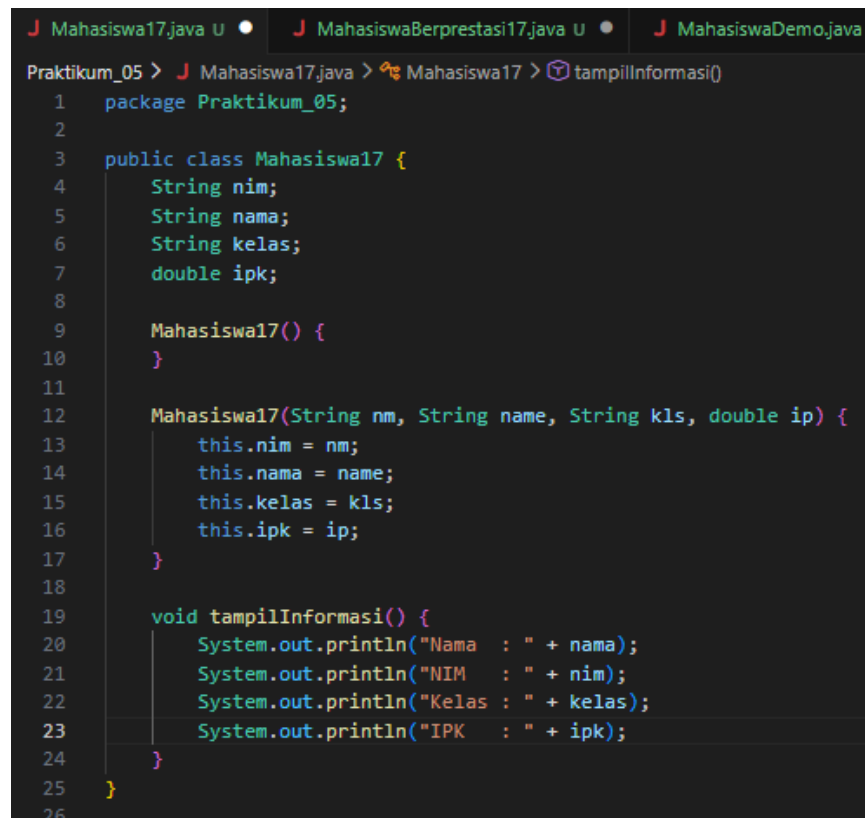
1. Buatlah class dengan nama Mahasiswa<No Presensi>.
2. Untuk lebih jelasnya class tersebut dapat dilihat pada potongan kode di bawah ini

```
public class Mahasiswa {
    String nim;
    String nama;
    String kelas;
    double ipk;

    // Konstruktor default
    Mahasiswa() {
    }

    // Konstruktor berparameter (dibuat ada yang nama var parameter inputnya sama ada yang tidak)
    Mahasiswa(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        ipk = ip;
        kelas = kls;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```



The screenshot shows an IDE with three tabs: 'Mahasiswa17.java U', 'MahasiswaBerprestasi17.java U', and 'MahasiswaDemo.java'. The active tab is 'Mahasiswa17.java U'. The code in the editor is as follows:

```
Praktikum_05 > J Mahasiswa17.java > Mahasiswa17 > tampilInformasi()
1 package Praktikum_05;
2
3 public class Mahasiswa17 {
4     String nim;
5     String nama;
6     String kelas;
7     double ipk;
8
9     Mahasiswa17() {
10    }
11
12     Mahasiswa17(String nm, String name, String kls, double ip) {
13         this.nim = nm;
14         this.nama = name;
15         this.kelas = kls;
16         this.ipk = ip;
17     }
18
19     void tampilInformasi() {
20         System.out.println("Nama : " + nama);
21         System.out.println("NIM : " + nim);
22         System.out.println("Kelas : " + kelas);
23         System.out.println("IPK : " + ipk);
24     }
25 }
26
```


3. Buat class **MahasiswaBerprestasi**<No Presensi> seperti di bawah ini!

```
public class MahasiswaBerprestasi {  
    Mahasiswa [] listMhs= new Mahasiswa [5];  
    int idx;  
}
```

4. Tambahkan method tambah() di dalam class tersebut! Method tambah() digunakan untuk menambahkan objek dari class Mahasiswa ke dalam atribut listMhs.

```
void tambah (Mahasiswa m){  
    if (idx<listMhs.length){  
        listMhs[idx]=m;  
        idx++;  
    }else {  
        System.out.println(x:"data sudah penuh");  
    }  
}
```

5. Tambahkan method tampil() di dalam class tersebut! Method tampil() digunakan untuk menampilkan semua data mahasiswa-mahasiswa yang ada di dalam class tersebut! Perhatikan penggunaan sintaks for yang agak berbeda dengan for yang telah dipelajari sebelumnya, meskipun secara konsep sebenarnya mirip.

```
void tampil (){  
    for (Mahasiswa m:listMhs){  
        m.tampilInformasi();  
        System.out.println(x:"-----");  
    }  
}
```

6. Tambahkan method bubbleSort() di dalam class tersebut!

```
void bubbleSort(){  
    for (int i=0; i<listMhs.length-1; i++){  
        for (int j=1; j<listMhs.length-i; j++){  
            if (listMhs[j].ipk>listMhs[j-1].ipk){  
                Mahasiswa tmp = listMhs[j];  
                listMhs[j]=listMhs[j-1];  
                listMhs[j-1]=tmp;  
            }  
        }  
    }  
}
```

```

J Mahasiswa17.java U • J MahasiswaBerprestasi17.java U • J MahasiswaDemo.java U
Praktikum_05 > J MahasiswaBerprestasi17.java > MahasiswaBerprestasi17 > bubbleSort()
1 package Praktikum_05;
2
3 public class MahasiswaBerprestasi17 {
4     Mahasiswa17 [] listMhs = new Mahasiswa17[5];
5     int idx;
6
7     void tambah(Mahasiswa17 m) {
8         if (idx < listMhs.length) {
9             listMhs[idx] = m;
10            idx++;
11        } else {
12            System.out.println(x: "Data sudah penuh!!");
13        }
14    }
15
16    void tampil() {
17        for (Mahasiswa17 m : listMhs) {
18            m.tampilInformasi();
19            System.out.println(x: "-----");
20        }
21    }
22
23    void bubbleSort() {
24        for (int i = 0; i < listMhs.length - 1; i++) {
25            for (int j = 1; j < listMhs.length - i; j++) {
26                if (listMhs[j].ipk > listMhs[j - 1].ipk) {
27                    Mahasiswa17 tmp = listMhs[j];
28                    listMhs[j] = listMhs[j - 1];
29                    listMhs[j - 1] = tmp;
30                }
31            }
32        }
33    }
34 }

```

7. Buat class **MahasiswaDemo**<No Presensi>, kemudian buatlah sebuah objek MahasiswaBerprestasi dan buatlah 5 objek mahasiswa kemudian tambahkan semua objek mahasiswa tersebut dengan memanggil fungsi tambah pada objek MahasiswaBerprestasi. Silakan dipanggil fungsi tampil() untuk melihat semua data yang telah dimasukan, urutkan data tersebut dengan memanggil fungsi bubbleSort() dan yang terakhir panggil fungsi tampil kembali.

```

public static void main(String[] args) {
    MahasiswaBerprestasi list = new MahasiswaBerprestasi();
    Mahasiswa m1 = new Mahasiswa(nm:"123", name:"Zidan",kls:"2A", ip:3.2);
    Mahasiswa m2 = new Mahasiswa(nm:"124", name:"Ayu",kls:"2A", ip:3.5);
    Mahasiswa m3 = new Mahasiswa(nm:"125", name:"Sofi",kls:"2A", ip:3.1);
    Mahasiswa m4 = new Mahasiswa(nm:"126", name:"Sita",kls:"2A", ip:3.9);
    Mahasiswa m5 = new Mahasiswa(nm:"127", name:"Miki",kls:"2A", ip:3.7);

    list.tambah(m1);
    list.tambah(m2);
    list.tambah(m3);
    list.tambah(m4);
    list.tambah(m5);

    System.out.println(x:"Data mahasiswa sebelum sorting: ");
    list.tampil();

    System.out.println(x:"Data Mahasiswa setelah sorting berdasarkan IPK (DESC) : ");
    list.bubbleSort();
    list.tampil();
}

```

```

J Mahasiswal7.java U • J MahasiswaBerprestasi17.java U • J MahasiswaDemo17.java U •
Praktikum_05 > J MahasiswaDemo17.java > MahasiswaDemo17 > main(String[])
1 package Praktikum_05;
2
3 public class MahasiswaDemo17 {
4     Run | Debug
5     public static void main(String[] args) {
6         MahasiswaBerprestasi17 list = new MahasiswaBerprestasi17();
7         Mahasiswal7 m1 = new Mahasiswal7(nm: "123", name: "Zidan", kls: "2A", ip: 3.2);
8         Mahasiswal7 m2 = new Mahasiswal7(nm: "124", name: "Ayu", kls: "2A", ip: 3.5);
9         Mahasiswal7 m3 = new Mahasiswal7(nm: "125", name: "Sofi", kls: "2A", ip: 3.1);
10        Mahasiswal7 m4 = new Mahasiswal7(nm: "126", name: "Sita", kls: "2A", ip: 3.9);
11        Mahasiswal7 m5 = new Mahasiswal7(nm: "127", name: "Miki", kls: "2A", ip: 3.7);
12
13        list.tambah(m1);
14        list.tambah(m2);
15        list.tambah(m3);
16        list.tambah(m4);
17        list.tambah(m5);
18
19        System.out.println(x: "Data sebelum sorting : ");
20        list.tampil();
21
22        System.out.println(x: "Data Mahasiswa setelah sorting berdasarkan IPK (DESC) : ");
23        list.bubbleSort();
24        list.tampil();
25    }
26 }

```

5.3.3 Verifikasi Hasil Percobaan

Data mahasiswa sebelum sorting:

Nama: Zidan
NIM: 123
IPK: 3.2
Kelas: 2A

Nama: Ayu
NIM: 124
IPK: 3.5
Kelas: 2A

Nama: Sofi
NIM: 125
IPK: 3.1
Kelas: 2A

Nama: Sita
NIM: 126
IPK: 3.9
Kelas: 2A

Nama: Miki
NIM: 127
IPK: 3.7
Kelas: 2A

Data Mahasiswa setelah sorting berdasarkan IPK (DESC) :

Nama: Sita
NIM: 126
IPK: 3.9
Kelas: 2A

Nama: Miki
NIM: 127
IPK: 3.7
Kelas: 2A

Nama: Ayu
NIM: 124
IPK: 3.5
Kelas: 2A

Nama: Zidan
NIM: 123
IPK: 3.2
Kelas: 2A

Nama: Sofi
NIM: 125
IPK: 3.1
Kelas: 2A

Data sebelum sorting :	Data Mahasiswa setelah sorting berdasarkan IPK (DESC) :
Nama : Zidan	Nama : Sita
NIM : 123	NIM : 126
Kelas : 2A	Kelas : 2A
IPK : 3.2	IPK : 3.9
-----	-----
Nama : Ayu	Nama : Miki
NIM : 124	NIM : 127
Kelas : 2A	Kelas : 2A
IPK : 3.5	IPK : 3.7
-----	-----
Nama : Sofi	Nama : Ayu
NIM : 125	NIM : 124
Kelas : 2A	Kelas : 2A
IPK : 3.1	IPK : 3.5
-----	-----
Nama : Sita	Nama : Zidan
NIM : 126	NIM : 123
Kelas : 2A	Kelas : 2A
IPK : 3.9	IPK : 3.2
-----	-----
Nama : Miki	Nama : Sofi
NIM : 127	NIM : 125
Kelas : 2A	Kelas : 2A
IPK : 3.7	IPK : 3.1
-----	-----

5.3.4 Pertanyaan

1. Perhatikan perulangan di dalam bubbleSort() di bawah ini:

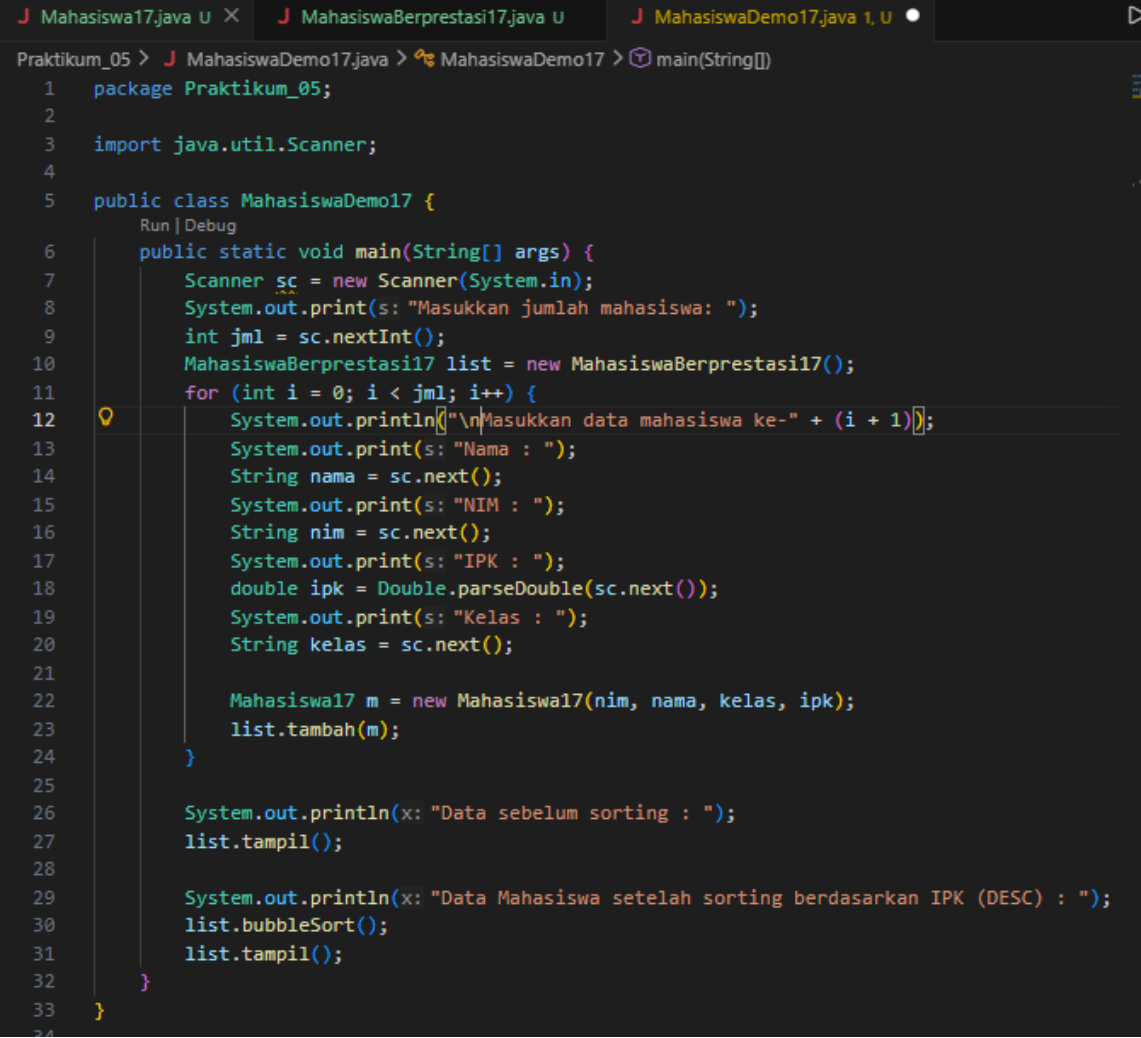
```
for (int i=0; i<listMhs.length-1; i++){
    for (int j=1; j<listMhs.length-i; j++){
```

- Mengapa syarat dari perulangan i adalah $i < \text{listMhs.length} - 1$?
 - Karena dalam algoritma Bubble Sort, untuk mengurutkan sejumlah N data, kita hanya memerlukan maksimal N-1 tahap. N-1 data sudah berada di posisi yang benar melalui proses bubble, maka data yang terakhir (ke-N) secara otomatis akan berada di posisi yang benar juga. Jadi, perulangan ke-N sudah tidak diperlukan lagi.
- Mengapa syarat dari perulangan j adalah $j < \text{listMhs.length} - i$?
 - Pada setiap tahap i, satu angka terbesar akan terkunci di posisi paling kanan. Dengan menggunakan -i, kita memerintahkan program agar tidak perlu lagi memeriksa data yang sudah berada di posisi benar (area yang sudah terurut di sebelah kanan). Semakin besar nilai i, semakin sedikit data yang perlu dibandingkan oleh perulangan j.
- Jika banyak data di dalam listMhs adalah 50, maka berapakah perulangan i akan berlangsung? Dan ada berapa Tahap bubble sort yang ditempuh?
 - Perulangan i akan berlangsung sebanyak 49 kali (0 sampai 48) karena syaratnya adalah $i < 50 - 1$ (alias $i < 49$), maka perulangan berhenti sebelum angka 49. Secara

total terdapat 49 tahap pengurutan untuk memastikan 50 data tersebut benar-benar urut.

2. Modifikasi program diatas dimana data mahasiswa bersifat dinamis (input dari keyboard) yang terdiri dari nim, nama, kelas, dan ipk!

- program :



```
Praktikum_05 > J MahasiswaDemo17.java > MahasiswaDemo17 > main(String[])
1 package Praktikum_05;
2
3 import java.util.Scanner;
4
5 public class MahasiswaDemo17 {
6     Run | Debug
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         System.out.print(s: "Masukkan jumlah mahasiswa: ");
10        int jml = sc.nextInt();
11        MahasiswaBerprestasi17 list = new MahasiswaBerprestasi17();
12        for (int i = 0; i < jml; i++) {
13            System.out.println("\nMasukkan data mahasiswa ke-" + (i + 1));
14            System.out.print(s: "Nama : ");
15            String nama = sc.next();
16            System.out.print(s: "NIM : ");
17            String nim = sc.next();
18            System.out.print(s: "IPK : ");
19            double ipk = Double.parseDouble(sc.next());
20            System.out.print(s: "Kelas : ");
21            String kelas = sc.next();
22
23            Mahasiswa17 m = new Mahasiswa17(nim, nama, kelas, ipk);
24            list.tambah(m);
25        }
26
27        System.out.println(x: "Data sebelum sorting : ");
28        list.tampil();
29
30        System.out.println(x: "Data Mahasiswa setelah sorting berdasarkan IPK (DESC) : ");
31        list.bubbleSort();
32        list.tampil();
33    }
34 }
```

- hasil :

```
Masukkan jumlah mahasiswa: 5
Masukkan data mahasiswa ke-1
Nama : Zidan
NIM : 123
IPK : 3.2
Kelas : 2A
Masukkan data mahasiswa ke-2
Nama : Ayu
NIM : 124
IPK : 3.5
Kelas : 2A
Masukkan data mahasiswa ke-3
Nama : Sofi
NIM : 125
IPK : 3.1
Kelas : 2A
Masukkan data mahasiswa ke-4
Nama : Sita
NIM : 126
IPK : 3.9
Kelas : 2A
Masukkan data mahasiswa ke-5
Nama : Miki
NIM : 127
IPK : 3.7
Kelas : 2A
```

Data sebelum sorting :

```
Nama : Zidan
NIM : 123
Kelas : 2A
IPK : 3.2
```

```
-----
Nama : Ayu
NIM : 124
Kelas : 2A
IPK : 3.5
```

```
-----
Nama : Sofi
NIM : 125
Kelas : 2A
IPK : 3.1
```

```
-----
Nama : Sita
NIM : 126
Kelas : 2A
IPK : 3.9
```

```
-----
Nama : Miki
NIM : 127
Kelas : 2A
IPK : 3.7
```

Data Mahasiswa setelah sorting berdasarkan IPK (DESC) :

```
Nama : Sita
NIM : 126
Kelas : 2A
IPK : 3.9
```

```
-----
Nama : Miki
NIM : 127
Kelas : 2A
IPK : 3.7
```

```
-----
Nama : Ayu
NIM : 124
Kelas : 2A
IPK : 3.5
```

```
-----
Nama : Zidan
NIM : 123
Kelas : 2A
IPK : 3.2
```

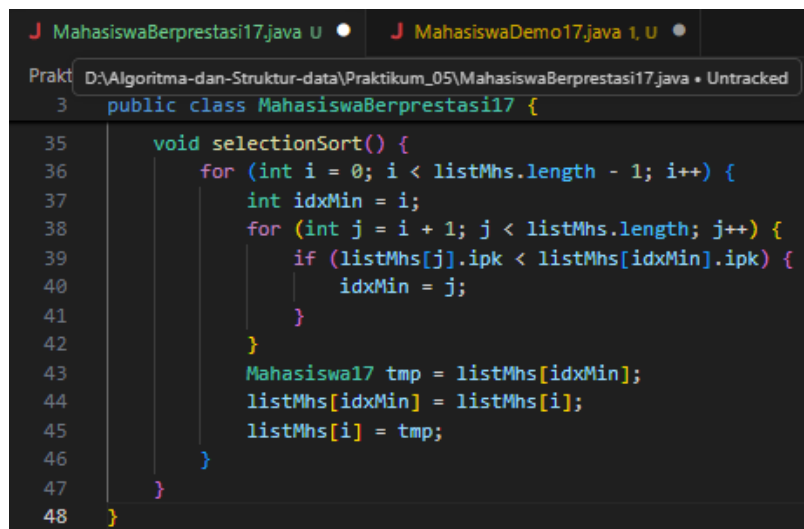
```
-----
Nama : Sofi
NIM : 125
Kelas : 2A
IPK : 3.1
```

5.3.5 Mengurutkan Data Mahasiswa Berdasarkan IPK (Selection Sort)

5.3.1 Langkah-langkah Percobaan.

1. Lihat kembali class MahasiswaBerprestasi, dan tambahkan method selectionSort() di dalamnya! Method ini juga akan melakukan proses sorting secara ascending, tetapi menggunakan pendekatan selection sort.

```
void selectionSort(){
    for (int i=0; i<listMhs.length-1; i++){
        int idxMin=i;
        for (int j=i+1; j<listMhs.length; j++){
            if (listMhs[j].ipk<listMhs[idxMin].ipk){
                idxMin=j;
            }
        }
        Mahasiswa tmp = listMhs[idxMin];
        listMhs[idxMin]=listMhs[i];
        listMhs[i]=tmp;
    }
}
```

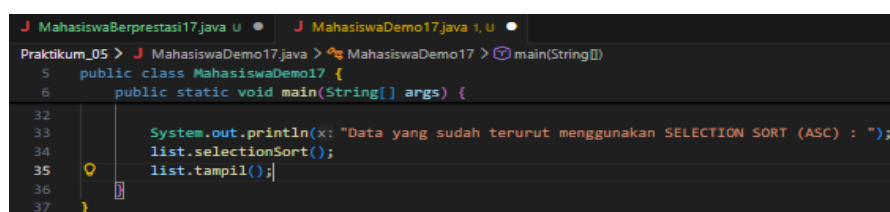


The screenshot shows a code editor with two tabs: 'MahasiswaBerprestasi17.java' and 'MahasiswaDemo17.java'. The active tab is 'MahasiswaBerprestasi17.java', which contains the following code:

```
public class MahasiswaBerprestasi17 {
    ...
    void selectionSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            int idxMin = i;
            for (int j = i + 1; j < listMhs.length; j++) {
                if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                    idxMin = j;
                }
            }
            Mahasiswa17 tmp = listMhs[idxMin];
            listMhs[idxMin] = listMhs[i];
            listMhs[i] = tmp;
        }
    }
}
```

2. Setelah itu, buka kembali class MahasiswaDemo, dan di dalam method main() tambahkan baris program untuk memanggil method selectionSort() tersebut, kemudian panggil method tampil() untuk menampilkan data yang sudah diurutkan!

```
System.out.println(x:"Data yang sudah terurut menggunakan SELECTION SORT (ASC)");
list.selectionSort();
list.tampil();
```



The screenshot shows a code editor with two tabs: 'MahasiswaBerprestasi17.java' and 'MahasiswaDemo17.java'. The active tab is 'MahasiswaDemo17.java', which contains the following code:

```
public class MahasiswaDemo17 {
    ...
    public static void main(String[] args) {
        ...
        System.out.println(x: "Data yang sudah terurut menggunakan SELECTION SORT (ASC) : ");
        list.selectionSort();
        list.tampil();
    }
}
```

3. Coba jalankan kembali class MahasiswaDemo, dan amati hasilnya! Apakah kini data mahasiswa telah tampilurut menaik berdasar ipk?

5.3.6 Verifikasi Hasil Percobaan

Pastikan output yang ditampilkan sudah benar seperti di bawah ini

Masukkan Data Mahasiswa ke-1	Data yang sudah terurut menggunakan SELECTION SORT (ASC)
NIM : 123	Nama: ila
Nama : Ali	NIM: 124
Kelas : 2B	Kelas: 2B
IPK : 3.9	IPK: 3.1
-----	-----
Masukkan Data Mahasiswa ke-2	Nama: udin
NIM : 124	NIM: 127
Nama : ila	Kelas: 2B
Kelas : 2B	IPK: 3.2
IPK : 3.1	-----
-----	Nama: tika
Masukkan Data Mahasiswa ke-3	NIM: 126
NIM : 125	Kelas: 2B
Nama : agus	IPK: 3.3
Kelas : 2B	-----
IPK : 3.6	Nama: agus
-----	NIM: 125
Masukkan Data Mahasiswa ke-4	Kelas: 2B
NIM : 126	IPK: 3.6
Nama : tika	-----
Kelas : 2B	Nama: Ali
IPK : 3.3	NIM: 123
-----	Kelas: 2B
Masukkan Data Mahasiswa ke-5	IPK: 3.9
NIM : 127	-----
Nama : udin	
Kelas : 2B	
IPK : 3.2	

Hasil

```
Masukkan data mahasiswa ke-1 :
NIM : 123
Nama : Ali
Kelas : 2B
IPK : 3.9
Masukkan data mahasiswa ke-2
NIM : 124
Nama : Ila
Kelas : 2B
IPK : 3.1
Masukkan data mahasiswa ke-3
NIM : 125
Nama : Agus
Kelas : 2B
IPK : 3.6
Masukkan data mahasiswa ke-4
NIM : 126
Nama : Tika
Kelas : 2B
IPK : 3.3
Masukkan data mahasiswa ke-5
NIM : 127
Nama : Udin
Kelas : 2B
IPK : 3.2
```

```
Data yang sudah terurut menggunakan SELECTION SORT (ASC) :
Nama : Ila
NIM : 124
Kelas : 2B
IPK : 3.1
-----
Nama : Udin
NIM : 127
Kelas : 2B
IPK : 3.2
-----
Nama : Tika
NIM : 126
Kelas : 2B
IPK : 3.3
-----
Nama : Agus
NIM : 125
Kelas : 2B
IPK : 3.6
-----
Nama : Ali
NIM : 123
Kelas : 2B
IPK : 3.9
-----
```


5.3.7 Pertanyaan

1. Di dalam method selection sort, terdapat baris program seperti di bawah ini:

```
int idxMin=i;
for (int j=i+1; j<listMhs.length; j++){
    if (listMhs[j].ipk<listMhs[idxMin].ipk){
        idxMin=j;
    }
}
```

Untuk apakah proses tersebut, jelaskan!

- berfungsi sebagai proses pencarian atau seleksi nilai terkecil (minimum) berdasarkan atribut tertentu (dalam hal ini adalah IPK) pada sekumpulan data mahasiswa yang belum terurut.
 - 1) Penentuan Acuan (int idxMin = i)
Sebelum menyisir data, algoritma menetapkan indeks i (posisi awal area yang belum terurut) sebagai pemegang nilai IPK terkecil sementara
 - 2) Proses Pemindaian (for loop)
Program melakukan scanning mulai dari posisi i + 1 sampai seluruh panjang array listMhs. Tujuannya adalah membandingkan IPK mahasiswa di posisi awal dengan mahasiswa-mahasiswa setelahnya.
 - 3) Seleksi Kondisi (if)
Di setiap langkah, program mengecek apakah listMhs[j].ipk lebih kecil daripada listMhs[idxMin].ipk. Jika ditemukan mahasiswa dengan IPK yang lebih rendah, maka posisi idxMin akan diperbarui ke indeks mahasiswa tersebut (j).
 - 4) etelah perulangan selesai, idxMin akan menyimpan indeks dari mahasiswa yang memiliki IPK paling rendah di area tersebut. Data inilah yang nantinya akan ditukar (swap) ke posisi depan untuk membentuk urutan.

5.4 Mengurutkan Data Mahasiswa Berdasarkan IPK Menggunakan Insertion Sort

5.4.1 Langkah-langkah Percobaan

1. Lihat kembali class MahasiswaBerprestasi, dan tambahkan method insertionSort() di dalamnya. Method ini juga akan melakukan proses sorting secara **ascending**, tetapi menggunakan pendekatan Insertion Sort.

```

void insertionSort(){
    for (int i=1; i<listMhs.length; i++){
        Mahasiswa temp = listMhs[i];
        int j=i;
        while (j>0 && listMhs[j-1].ipk>temp.ipk){
            listMhs[j]=listMhs[j-1];
            j--;
        }
        listMhs[j]=temp;
    }
}

```

```

48
49
50 void insertion() {
51     for (int i = 1; i < listMhs.length; i++) {
52         Mahasiswa17 temp = listMhs[i];
53         int j = i;
54         while (j > 0 && listMhs[j-1].ipk > temp.ipk) {
55             listMhs[j] = listMhs[j-1];
56             j--;
57         }
58         listMhs[j] = temp;
59     }
60 }

```

- Setelah itu, buka kembali class MahasiswaDemo, dan di dalam method main() tambahkan baris program untuk memanggil method insertionSort() dan tampil () tersebut!

```

System.out.println(x:"Data yang sudah terurut menggunakan INSERTION SORT (ASC)");
list.insertionSort();
list.tampil();

```

```

5 public class MahasiswaDemo17 {
6     public static void main(String[] args) {
37         System.out.println(x: "Data yang sudah terurut menggunakan INSERTION SORT (ASC) : ");
38         list.insertionSort();
39         list.tampil();
40     }
41 }

```

- Coba jalankan kembali class MahasiswaDemo, dan amati hasilnya! Apakah kini data mahasiswa telah tampil urut menaik berdasar ipk?

5.4.2 Verifikasi Hasil Percobaan

Pastikan output yang ditampilkan sudah benar seperti di bawah ini

Masukkan Data Mahasiswa ke-1	Data yang sudah terurut menggunakan INSERTION SORT (ASC)
NIM : 111	Nama: dika
Nama : ayu	NIM: 222
Kelas : 2c	Kelas: 2c
IPK : 3.7	IPK: 3.0
-----	-----
Masukkan Data Mahasiswa ke-2	Nama: susi
NIM : 222	NIM: 444
Nama : dika	Kelas: 2c
Kelas : 2c	IPK: 3.1
IPK : 3.0	-----
-----	Nama: yayuk
Masukkan Data Mahasiswa ke-3	NIM: 555
NIM : 333	Kelas: 2c
Nama : ila	IPK: 3.4
Kelas : 2c	-----
IPK : 3.8	Nama: ayu
-----	NIM: 111
Masukkan Data Mahasiswa ke-4	Kelas: 2c
NIM : 444	IPK: 3.7
Nama : susi	-----
Kelas : 2c	Nama: ila
IPK : 3.1	NIM: 333
-----	Kelas: 2c
Masukkan Data Mahasiswa ke-5	IPK: 3.8
NIM : 555	-----
Nama : yayuk	
Kelas : 2c	
IPK : 3.4	

Hasil :

Masukkan data mahasiswa ke-1	Data yang sudah terurut menggunakan INSERTION SORT (ASC)
NIM : 111	Nama : Dika
Nama : Ayu	NIM : 222
Kelas : 2c	Kelas : 2c
IPK : 3.7	IPK : 3.0
Masukkan data mahasiswa ke-2	-----
NIM : 222	Nama : susi
Nama : Dika	NIM : 444
Kelas : 2c	Kelas : 2c
IPK : 3.0	IPK : 3.1
Masukkan data mahasiswa ke-3	-----
NIM : 333	Nama : Yayuk
Nama : Ila	NIM : 555
Kelas : 2c	Kelas : 2c
IPK : 3.8	IPK : 3.4
Masukkan data mahasiswa ke-4	-----
NIM : 444	Nama : Ayu
Nama : susi	NIM : 111
Kelas : 2c	Kelas : 2c
IPK : 3.1	IPK : 3.7
Masukkan data mahasiswa ke-5	-----
NIM : 555	Nama : Ila
Nama : Yayuk	NIM : 333
Kelas : 2c	Kelas : 2c
IPK : 3.4	IPK : 3.8

5.4.3 Pertanyaan

1. Ubahlah fungsi pada InsertionSort sehingga fungsi ini dapat melaksanakan proses sorting dengan cara descending.

```
J MahasiswaBerprestasi17.java U X J MahasiswaDemo17.java 1, U ●
Praktikum_05 > J MahasiswaBerprestasi17.java > MahasiswBerprestasi17
3 public class MahasiswaBerprestasi17 {
49     void insertionDescending() {
50         for (int i = 1; i < listMhs.length; i++) {
51             Mahasiswa17 temp = listMhs[i];
52             int j = i;
53             while (j > 0 && listMhs[j-1].ipk < temp.ipk) {
54                 listMhs[j] = listMhs[j-1];
55                 j--;
56             }
57             listMhs[j] = temp;
58         }
59     }
60 }
```

```
Data yang sudah terurut menggunakan INSERTION DESCENDING SORT (ASC) :
Nama : Ila
NIM : 333
Kelas : 2c
IPK : 3.8
-----
Nama : Ayu
NIM : 111
Kelas : 2c
IPK : 3.7
-----
Nama : Yayuk
NIM : 555
Kelas : 2c
IPK : 3.4
-----
Nama : Susi
NIM : 444
Kelas : 2c
IPK : 3.1
-----
Nama : Dika
NIM : 222
Kelas : 2c
IPK : 3.0
-----
```

5.5 Latihan Praktikum

Perhatikan class diagram dibawah ini:

Dosen
kode: String nama: String jenisKelamin: Boolean usia: int
Dosen(kd: String, name: String, jk: Boolean, age: int) tampil(): void

DataDosen
dataDosen: Dosen[10] idx: int
tambah(dsn: Dosen): void tampil(): void SortingASC(): void sortingDSC():void insertionSort():void

Berdasarkan class diagram diatas buatlah menu dikelas main dengan pilihan menu:

1. Tambah data digunakan untuk menambahkan data dosen
2. Tampil data digunakan untuk menampilkan data seluruh dosen
3. Sorting ASC digunakan untuk mengurutkan data dosen berdasarkan usia dimulai dari dosen termuda ke dosen tertua menggunakan bubble Sort.
4. Sorting DSC digunakan untuk mengurutkan data dosen berdasarkan usia dimulai dari tertua ke dosen termuda dapat menggunakan algoritma selection sort atau insertion sort.

Program :

```

J Dosen17.java U X  J dataDosen17.java U  J DosenMain17.java U
Praktikum_05 > J Dosen17.java > Dosen17
1  package Praktikum_05;
2
3  public class Dosen17 {
4      String kode;
5      String nama;
6      boolean jenisKelamin;
7      int usia;
8
9      Dosen17(String kd, String name, boolean jk, int age) {
10         kode = kd;
11         nama = name;
12         jenisKelamin = jk;
13         usia = age;
14     }
15
16     void tampil() {
17         String jk = (jenisKelamin) ? "Pria" : "Wanita";
18         System.out.printf(format: "| %-6s | %-20s | %-12s | %-4d |\n", kode, nama, jk, usia);
19     }
20 }

```

J Dosen17.java u J dataDosen17.java u X J DosenMain17.java u

Praktikum_05 > J dataDosen17.java > dataDosen17 > tambah(Dosen17)

```
1 package Praktikum_05;
2
3 public class dataDosen17 {
4     Dosen17[] dataDosen = new Dosen17[10];
5     int idx = 0;
6
7     void tambah(Dosen17 dsn) {
8         if (idx < dataDosen.length) {
9             dataDosen[idx] = dsn;
10            idx++;
11            System.out.println(x: "Data dosen berhasil ditambahkan.");
12        } else {
13            System.out.println(x: "-----");
14            System.out.println(x: "DATA SUDAH PENUH!");
15            System.out.println(x: "-----");
16        }
17    }
18
19    void tampilSeluruhData() {
20        if (idx == 0) {
21            System.out.println(x: "Data masih kosong!");
22            return;
23        }
24        System.out.println(x: "=====");
25        System.out.printf(format: "| %-6s | %-20s | %-12s | %-4s |\n", ...args: "KODE", "NAMA", "GENDER", "USIA");
26        System.out.println(x: "=====");
27        for (int i = 0; i < idx; i++) {
28            dataDosen[i].tampil();
29        }
30        System.out.println(x: "=====");
31    }
32
33    // Bubble Sort - Usia Ascending
34    void bubbleSortUsiaAscending() {
35        for (int i = 0; i < idx - 1; i++) {
36            for (int j = 1; j < idx - i; j++) {
37                if (dataDosen[j].usia < dataDosen[j - 1].usia) {
38                    Dosen17 temp = dataDosen[j];
39                    dataDosen[j] = dataDosen[j - 1];
40                    dataDosen[j - 1] = temp;
41                }
42            }
43        }
44        System.out.println(x: "Data telah diurutkan Ascending (Bubble Sort).");
45    }
46
47    // Selection Sort - Usia Descending
48    void selectionSortUsiaDescending() {
49        for (int i = 0; i < idx - 1; i++) {
50            int max_idx = i;
51            for (int j = i + 1; j < idx; j++) {
52                if (dataDosen[j].usia > dataDosen[max_idx].usia) {
53                    max_idx = j;
54                }
55            }
56            Dosen17 temp = dataDosen[max_idx];
57            dataDosen[max_idx] = dataDosen[i];
58            dataDosen[i] = temp;
59        }
60        System.out.println(x: "Data telah diurutkan Descending (Selection Sort).");
61    }
62 }
```

```
J Dosen17.java U    J dataDosen17.java U    J DosenMain17.java U X
Praktikum_05 > J DosenMain17.java > DosenMain17
1  package Praktikum_05;
2
3  import java.util.Scanner;
4
5  public class DosenMain17 {
    Run | Debug
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8          dataDosen17 pengelola = new dataDosen17();
9          int pilih;
10
11          do {
12              System.out.println(x: "\n=== MENU MANAJEMEN DATA DOSEN ===");
13              System.out.println(x: "1. Tambah Data Dosen");
14              System.out.println(x: "2. Tampilkan Seluruh Data Dosen");
15              System.out.println(x: "3. Urutkan Usia Ascending (Bubble Sort)");
16              System.out.println(x: "4. Urutkan Usia Descending (Selection Sort)");
17              System.out.println(x: "5. Keluar");
18              System.out.print(s: "Pilih menu (1-5): ");
19              pilih = sc.nextInt();
20
21              switch (pilih) {
22                  case 1:
23                      System.out.print(s: "Kode Dosen      : "); String kd = sc.next();
24                      System.out.print(s: "Nama Dosen      : "); sc.nextLine();
25                      String nama = sc.nextLine();
26                      System.out.print(s: "Jenis Kelamin (P/W): "); char jkChar = sc.next().charAt(index: 0);
27                      boolean jk = (jkChar == 'P' || jkChar == 'p');
28                      System.out.print(s: "Usia            : "); int usia = sc.nextInt();
29
30                      Dosen17 d = new Dosen17(kd, nama, jk, usia);
31                      pengelola.tambah(d);
32                      break;
33                  case 2:
34                      pengelola.tampilSeluruhData();
35                      break;
36                  case 3:
37                      pengelola.bubbleSortUsiaAscending();
38                      pengelola.tampilSeluruhData();
39                      break;
40                  case 4:
41                      pengelola.selectionSortUsiaDescending();
42                      pengelola.tampilSeluruhData();
43                      break;
44                  case 5:
45                      System.out.println(x: "Terima kasih!");
46                      break;
47                  default:
48                      System.out.println(x: "Pilihan tidak valid!");
49              }
50          } while (pilih != 5);
51          sc.close();
52      }
53  }
```

Hasil :

```
=== MENU MANAJEMEN DATA DOSEN ===
1. Tambah Data Dosen
2. Tampilkan Seluruh Data Dosen
3. Urutkan Usia Ascending (Bubble Sort)
4. Urutkan Usia Descending (Selection Sort)
5. Keluar
Pilih menu (1-5): 1
Kode Dosen    : ds1
Nama Dosen    : Arul
Jenis Kelamin (P/W): P
Usia          : 32
Data dosen berhasil ditambahkan.
```

```
=== MENU MANAJEMEN DATA DOSEN ===
1. Tambah Data Dosen
2. Tampilkan Seluruh Data Dosen
3. Urutkan Usia Ascending (Bubble Sort)
4. Urutkan Usia Descending (Selection Sort)
5. Keluar
Pilih menu (1-5): 1
Kode Dosen    : ds2
Nama Dosen    : Nurul
Jenis Kelamin (P/W): W
Usia          : 38
Data dosen berhasil ditambahkan.
```

```
=== MENU MANAJEMEN DATA DOSEN ===
1. Tambah Data Dosen
2. Tampilkan Seluruh Data Dosen
3. Urutkan Usia Ascending (Bubble Sort)
4. Urutkan Usia Descending (Selection Sort)
5. Keluar
Pilih menu (1-5): 1
Kode Dosen    : ds3
Nama Dosen    : Ahmad
Jenis Kelamin (P/W): P
Usia          : 42
Data dosen berhasil ditambahkan.
```

```
=== MENU MANAJEMEN DATA DOSEN ===
1. Tambah Data Dosen
2. Tampilkan Seluruh Data Dosen
3. Urutkan Usia Ascending (Bubble Sort)
4. Urutkan Usia Descending (Selection Sort)
5. Keluar
Pilih menu (1-5): 2
```

KODE	NAMA	GENDER	USIA
ds1	Arul	Pria	32
ds2	Nurul	Wanita	38
ds3	Ahmad	Pria	42

```
=== MENU MANAJEMEN DATA DOSEN ===
1. Tambah Data Dosen
2. Tampilkan Seluruh Data Dosen
3. Urutkan Usia Ascending (Bubble Sort)
4. Urutkan Usia Descending (Selection Sort)
5. Keluar
Pilih menu (1-5): 3
Data telah diurutkan Ascending (Bubble Sort).
```

KODE	NAMA	GENDER	USIA
ds1	Arul	Pria	32
ds2	Nurul	Wanita	38
ds3	Ahmad	Pria	42

```
=== MENU MANAJEMEN DATA DOSEN ===
1. Tambah Data Dosen
2. Tampilkan Seluruh Data Dosen
3. Urutkan Usia Ascending (Bubble Sort)
4. Urutkan Usia Descending (Selection Sort)
5. Keluar
Pilih menu (1-5): 4
Data telah diurutkan Descending (Selection Sort).
```

KODE	NAMA	GENDER	USIA
ds3	Ahmad	Pria	42
ds2	Nurul	Wanita	38
ds1	Arul	Pria	32

```
=== MENU MANAJEMEN DATA DOSEN ===
1. Tambah Data Dosen
2. Tampilkan Seluruh Data Dosen
3. Urutkan Usia Ascending (Bubble Sort)
4. Urutkan Usia Descending (Selection Sort)
5. Keluar
Pilih menu (1-5): 5
Terima kasih!
```